

Projeto – fase 3



1. Introdução

O projeto da disciplina de Aplicações Distribuídas está dividido em três fases. A fase 3 evolui o sistema desenvolvido nas fases 1 e 2. A lógica de negócio mantém-se a mesma. Isto significa que a interface com o utilizador do lado do cliente mantém-se inalterada. O servidor atual da fase 2 é suportado por uma solução de multiplexação de I/O e sockets.

Na 3ª e última fase do projeto, iremos tornar a comunicação segura utilizando os protocolos SSL/TLS e suportar tolerância a falhas por meio da replicação do estado do servidor, seguindo o modelo Chain Replication (Replicação em Cadeia) [1] e usando o serviço de coordenação ZooKeeper [2]. De forma genérica, vai ser preciso:

- Implementar comunicação segura utilizando os protocolos SSL/TLS;
- Implementar a coordenação de servidores no ZooKeeper, de forma a suportar o modelo de replicação em cadeia (Chain Replication);

Alterar o funcionamento do servidor para:

- Procurar no ZooKeeper o servidor seguinte (sucessor) na cadeia de replicação;
- Depois de executar uma operação de escrita, enviá-la para o servidor seguinte (sucessor) de forma a propagar a replicação;
- Fazer watch no ZooKeeper de forma a ser notificado de alterações na cadeia de replicação e ligar-se ao seu novo sucessor, caso este tenha mudado.

Alterar o funcionamento do cliente para:

- Procurar no ZooKeeper os servidores que estão na cabeça e na cauda da cadeia;
- Implementar comunicação segura utilizando os protocolos SSL/TLS;
- Mandar as operações de escrita para o servidor que está na cabeça da cadeia;
- Mandar as operações de leitura para o servidor que está na cauda da cadeia;
- Fazer watch no ZooKeeper de forma a ser notificado de alterações na cadeia de replicação e ligar-se a novos servidores (na cabeça ou na cauda da cadeia), se estes tiverem mudado.

Dada a complexidade de uma aplicação real do tipo MarketCenter, espera-se uma grande fiabilidade do servidor e do cliente, sendo, para tal, necessário tratar todas as condições de erro e garantir uma gestão correta da memória. É importante notar que, em sistemas cliente-servidor os servidores ficam em funcionamento permanente durante muito tempo (dias, meses ou até anos) e não é suposto que parem (crash) por causa de erros.

2. Descrição detalhada

O objetivo específico da 3ª Fase do Projeto é desenvolver um sistema de Replicação em Cadeia (Chain Replication) [1] com múltiplos clientes e servidores. Para tal, para além de aproveitarem o código desenvolvido nas fases anteriores do projeto, os alunos devem fazer o uso de novas técnicas ensinadas nas aulas, incluindo o serviço ZooKeeper [2] para coordenação de sistemas distribuídos.

A figura a seguir ilustra a arquitetura final do sistema a desenvolver.

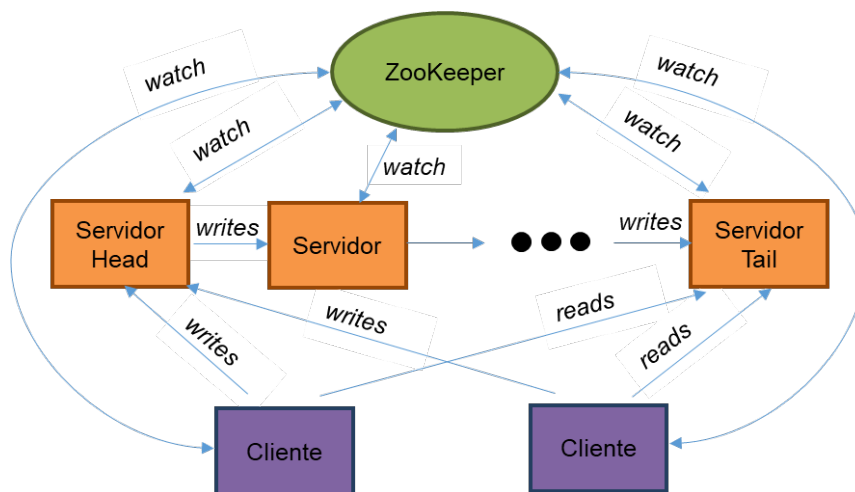


Figura 1 - Arquitetura geral da 3ª Fase do Projeto

Num modelo de replicação em cadeia, os servidores ligam-se entre si, formando uma sequência; ou seja, cada servidor comunica apenas com o seu sucessor na cadeia. Por outro lado, os clientes têm de ter duas ligações: uma ao servidor na cabeça da cadeia e outra ao servidor na cauda.

Para garantir que todos os servidores têm cópias iguais (isto é, garantem a consistência dos dados), os clientes têm de enviar todas as operações de escrita para o servidor que está à cabeça da cadeia, sendo estas depois propagadas de forma síncrona (isto é, ficando à espera da resposta) pelos servidores seguintes, até chegarem ao servidor que está na cauda. Assim, quando um servidor recebe uma escrita, executa-a localmente e depois executa-a remotamente no seu sucessor (caso não esteja na cauda). As duas escritas (local e remota) têm de ser feitas de forma atômica, ou seja, enquanto não forem feitas as duas, mais nenhuma operação de escrita pode ser realizada pelo servidor.

Por outro lado, os clientes enviam as operações de leitura para o servidor que está na cauda.

A ideia é que existam sempre pelo menos dois servidores ativos, para que a falha de um servidor possa ser tolerada. Se um servidor falhar ficará sempre pelo menos um servidor ativo, não se perdendo a informação do sistema. Deverá ser possível lançar novos servidores, fazendo com que estes se integrem na cadeia, ficando na cauda. Para se integrar, um novo servidor tem de começar por obter uma cópia atual do sistema e, só depois, poderá começar a atender aos pedidos dos clientes.

Neste projeto, vamos considerar que as falhas dos servidores apenas podem acontecer quando não existem operações pendentes nos servidores (ou seja, não há clientes a fazer operações quando ocorre uma falha) e vamos também considerar que quando um novo servidor é iniciado também não há operações a serem executadas. Graças a estas considerações, a solução a implementar acaba por simplificar-se.

2.1 ZooKeeper

Um elemento central na arquitetura desta fase do projeto é o ZooKeeper, que será usado para armazenar a informação sobre os servidores ativos e avisará todos os nós, clientes ou servidores sempre que algum servidor for adicionado ou falhar. Assim, o ZooKeeper permitirá manter uma visão completa e atualizada do sistema, não só sobre os servidores ativos, mas também sobre seus endereços IP e portos. Quando se iniciam os clientes, apenas será necessário passar como argumento a localização do ZooKeeper (IP:porto do ZooKeeper). Tanto os clientes como os servidores terão de se ligar e manter uma ligação ao ZooKeeper, interagindo com este através das funções disponibilizadas na API do ZooKeeper.

O ZooKeeper pode ser descrito como um serviço (eventualmente replicado, embora neste projeto se utilize apenas um servidor ZooKeeper) que armazena a informação organizada de forma hierárquica em nós que são designados ZNodes.

Neste projeto, vamos utilizar uma ordenação global entre os servidores. Isto garante que existem sempre um (e apenas um) servidor head e um tail. Para tal, devemos criar os node como ZNodes sequenciais. Assim, o ZooKeeper atribui um número de sequência único e crescente a cada novo /node que é criado. Quando se obtém a listagem de filhos de /chain, podemos ordenar os mesmos por ordem lexicográfica.

Como também pretendemos lidar com falhas dos servidores e detetar as mesmas de forma automática, vamos adicionalmente criar os /node como ZNodes efêmeros. Assim, se um servidor falhar, o ZooKeeper deteta que a ligação entre os dois foi interrompida e remove o seu /node da lista de filhos de /chain, notificando todos os servidores e clientes

que fizeram watch aos filhos de /chain. Isto significa que todos os servidores e clientes, quando são iniciados e se ligam ao ZooKeeper, devem indicar uma função que servirá como callback para receber estas notificações. E quando chamam uma função da API para ler dados do ZooKeeper, devem indicar ao ZooKeeper que querem ser notificados se a informação lida for alterada (ou seja, colocam o watch no ZNode lido).

2.2 Mudanças a efetuar no servidor

O servidor passa a guardar:

- uma ligação ao ZooKeeper;
- o identificador do seu /node no ZooKeeper;
- o identificador do /node do próximo servidor na cadeia de replicação, assim como uma socket para a comunicação com o mesmo.

Novos passos a implementar na lógica do servidor quando este é iniciado:

- Ligar ao ZooKeeper;
- Criar um ZNode efêmero sequencial no ZooKeeper, filho de /chain;
- Guardar o id atribuído ao ZNode pelo ZooKeeper;
- Obter e fazer watch aos filhos de /chain;
- Ver se existe um nó sucessor de entre os filhos de /chain, ou seja, um nó com o identificador mais alto a seguir ao próprio identificador. Se existir, obter o conteúdo do Znode desse servidor (ou seja, IP:porto), configurando-o como sucessor. Caso contrário, o sucessor ficará a NULL, o que significa que este servidor será a cauda da cadeia;
- Ver se existe um nó antecessor entre os filhos de /chain, ou seja, o servidor com o identificador mais baixo antes do próprio identificador. Se existir, obter o conteúdo do Znode desse servidor (ou seja, IP:porto), conectando-se a ele temporariamente para obter uma cópia do estado do servidor. No final, terminar a ligação ao servidor antecessor. Este passo serve para a sincronização do estado da aplicação.

Adicionalmente:

- Quando a função de callback é chamada devido ao watch de filhos de /chain ter sido ativado, voltar a obter a lista de filhos de /chain para saber se o nó sucessor foi alterado novamente. Caso tenha sido (porque o sucessor falhou, ou porque não havia sucessor e passou a haver), deve-se estabelecer uma ligação a este novo servidor e atualizar sucessor;
- Quando receber uma operação de escrita, executar a operação localmente e, se não estiver na cauda, executar também remotamente no servidor sucessor. Estas duas operações (escrita local e escrita no sucessor) têm de ser feitas de forma atômica, ou seja, sem executar outras escritas enquanto estas não forem concluídas.

2.3 Mudanças a efetuar no cliente

O cliente passa a ligar-se ao ZooKeeper e a dois servidores: o servidor head e o tail da cadeia de replicação. Se existir apenas um servidor ativo, este será tanto head como a tail e os clientes terão duas ligações ao mesmo servidor.

Novos passos a implementar na lógica do cliente quando este é iniciado:

- Ligar ao ZooKeeper;
- Obter e fazer watch aos filhos de /chain;
- Dos filhos de /chain, obter o IP:porto do servidor registado com o identificador mais baixo e do servidor que tem o identificador mais alto, guardá-los como head e tail respetivamente, ligando-se a eles;

Adicionalmente:

Quando a função de callback é chamada devido ao watch de filhos de /chain ter sido ativado, voltar a obter a lista de filhos de /chain para saber se algum dos servidores head ou tail foi alterado, ligando-se ao novo servidor caso necessário;

O cliente passa a enviar, de forma segura utilizando os protocolos SSL/TLS:

- Pedidos de escrita para o servidor head, que serão depois propagados por toda a cadeia;
- Pedidos de leitura para o servidor tail.

Neste modelo, o IP e porto introduzidos pelo utilizador no comando do cliente passam a ser o IP e porto do ZooKeeper apenas.

3. Lógica de Negócio e Tratamento de Erros

Os alunos devem manter a lógica de negócio e a interface com o utilizador estipulada desde a fase 1. Deve ser implementado um tratamento de erros avançado.

4. Estruturação de Código

Aplicam-se as mesmas regras das Fases 1 e 2, ao nível da estruturação de código.

5. Entrega

A entrega da fase 3 do projeto tem de ser feita de acordo com as seguintes regras:

- Colocar todos os ficheiros do projeto, bem como um ficheiro README, num único ficheiro ZIP. O nome do ficheiro será AD-XX-fase3.zip (XX é o nome do grupo).
- Submeter o ficheiro AD-XX-fase3.zip na página da disciplina no Moodle de Ciências@ULisboa, utilizando a atividade disponibilizada para tal. Apenas **um** dos elementos do grupo deve submeter.

Os programas são testados no ambiente Linux dos laboratórios das aulas. Qualquer trabalho entregue que não execute corretamente no ambiente dos laboratórios não será considerado para avaliação. Recomenda-se que usem os laboratórios para o desenvolvimento do projeto e que testem os programas nesse mesmo ambiente.

O prazo de entrega é dia 15/5/2026, até às 23:59 horas.

Após esta data, a submissão do trabalho através do Moodle deixará de ser permitida.

Não são aceites entregas por email.

6. Plágio

Aplicam-se as mesmas regras da Fase 1 a nível de questões de plágio. Ou seja, não é permitido aos alunos partilhar códigos com soluções, ainda que parciais, de qualquer parte do projeto com outros alunos (nem através do Fórum da disciplina, nem por qualquer outro meio). Além disso, todos os códigos serão submetidos a um verificador de plágio. Caso seja encontrada alguma irregularidade, os projetos de todos os alunos envolvidos serão anulados e o caso será reportado aos órgãos responsáveis em Ciências@ULisboa.

Chamamos a atenção para o facto de as plataformas generativas baseadas em Inteligência Artificial (e.g., o ChatGPT e o GitHub Co-Pilot) (1) gerarem um número limitado de soluções diferentes para o mesmo problema, (2) não resolverem corretamente as alíneas descritas no projeto, e (3) incluírem padrões característicos deste tipo de ferramenta, os quais podem vir a ser detetáveis pelos verificadores de plágio. Desta forma, recomendamos fortemente que os alunos não submetam trechos de código gerados por este tipo de ferramenta a fim de evitar riscos desnecessários.

Por fim, é responsabilidade de cada aluno garantir que a sua *home*, as suas diretorias e os seus ficheiros de código estão protegidos contra a leitura de outras pessoas (que não o utilizador dono dos mesmos). Por exemplo, se os ficheiros estiverem gravados na sua área de aluno nos servidores de Ciências@ULisboa, então todos os itens mencionados anteriormente devem ter as permissões de acesso 700. Se os ficheiros estiverem no GitHub, garantam que o conteúdo do vosso repositório não esteja visível publicamente. Caso contrário, a sua participação num eventual plágio será considerada ativa.

7. Bibliografia

[1] R. V. Renesse and F. B. Schneider. Chain Replication for Supporting High Throughput and Availability. OSDI. Vol. 4. No. 91–104. 2004.

[2] <https://zookeeper.apache.org/>